COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

Cloud Specialist [D68444-15-2025-23]

Unità Formativa : Programmazione Python

Docente: Massimo PAPA

Titolo argomento: Esercizi sulle classi (OOP) (3)

Intervento realizzato da ITS TECNOLOGIE DELL'INFORMAZIONE E DELLA COMUNICAZIONE

Esercizi sulle classi (OOP)

Indicazioni sulla consegna **3**
Struttura del codice sorgente 4
Esercizi sulle classi (OOP) **6**
Primo Esercizio 6
Secondo Esercizio 6
Terzo Esercizio 6
Esercizi sull'ereditarietà **7**
Quarto Esercizio 7
Quinto Esercizio 7
Esercizi sul polimorfismo e sull'import **8**
Sesto Esercizio 8
Settimo Esercizio 9
Ottavo Esercizio 9

Indicazioni sulla consegna

Implementare gli algoritmi risolutivi dei seguenti esercizi codificandoli in linguaggio python. Partire analizzando il problema seguendo la seguente traccia:

- Quali sono gli input del problema?
- Quali sono gli output?
- Suddividi il problema in problemi più semplici e se lo ritieni opportuno descrivi la soluzione di ogni sottoproblema con un flow-chart
- Andrai a rappresentare ogni algoritmo che risolve un sottoproblema come una funzione
- La funzione restituisce un valore? Se si di che tipo?
- Ogni funzione accetta una lista di parametri formali? Se si di che tipo?
- Esegui la codifica partendo dal programma principale, al suo interno vai a richiamare le funzioni che poi andrai successivamente a definire.
- Continua la codifica dichiarando e definendo tutte le funzioni prima del programma principale che richiama tutte le altre. In testa alle funzioni scrivi un commento che descrive la funzione stessa , cosa restituisce e quali sono i parametri formali.
- Scrivi la sezione dell'inizializzazione variabili scrivendo in un commento il significa della variabile stessa
- Verifica la codifica utilizzando input di test, cercando di provare anche i casi limite.

Struttura del codice sorgente

Relativamente alle indicazioni di scrittura del codice, utilizza il seguente schema generale:

```
'''
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Testo esercizio

'''

##
## Funzioni:
##


def fun(param1: int, param2: float) -> int:
  '''
     Funzione: fun
      Template per costruire le funzioni
      Parametri formali:
          int Param1 -> descrizione Param1
          float Param2 -> descrizione Param2
      Valore di ritorno:
          int -> descrizione valore di ritorno
  '''
  retValue = 5 # Valore di ritorno della funzione

  return retValue


'''
    Programma principale
    Descrizione sintetica funzionalità
    del programma principale.
'''

# Sezione di input Dati

# Inizializzazioni variabili

# Elaborazione
```

```
# Eventuali sotto processi di Elaborazione

# Sezione di output
```

Esercizi sulle classi (OOP)

Primo Esercizio

Creare una classe Insegnante con attributi nome, età e stipendio, dove stipendio deve essere un attributo privato.

Costruire tutti i metodi getter e setter per gli attributi (anche per quelli pubblici)

Effettuare l'overriding del metodo __str__ in maniera tale che restituisca gli attributi nome e età.

Provare la classe istanziando almeno due oggetti.

Secondo Esercizio

Creare una classe Rettangolo con attributi base e altezza. Costruire tutti i metodi setter e getter per gli attributi. Aggiungere i metodi per il calcolo dell'area e del perimetro.

Implementare un metodo di nome: "contiene" che ha come parametro un oggetto rettangolo. Tale metodo deve restituire true se il rettangolo oggetto chiamante contiene il rettangolo oggetto parametro, false se non lo contiene. Un rettangolo "contiene" un altro quando la sua altezza e la sua base sono maggiori rispettivamente della base e dell'altezza del secondo rettangolo.

Terzo Esercizio

1 - Creare una classe Calcolo con un costruttore di default (senza parametri) che consenta di eseguire vari calcoli su numeri interi.

2 - Creare un metodo chiamato Factorial() che permetta di calcolare il fattoriale di un intero. Testare il metodo istanziando la classe.

3 - Creare un metodo chiamato Sum() che consenta di calcolare la somma dei primi n interi 1 + 2 + 3 + .. + n. Prova questo metodo.

4 - Creare un metodo tableMult() che crea e visualizza la tabellina di un dato intero. Quindi creare un metodo allTablesMult() per visualizzare tutte le tabelline di numeri interi 1, 2, 3, ..., 9.

Esercizi sull'ereditarietà

Quarto Esercizio

Si definisca una classe Persona che abbia i seguenti attributi:

- nome
- indirizzo
- età

Tale classe contiene i seguenti metodi: il costruttore, l'overriding del metodo __str__ e tutti i metodi getter e setter degli attributi.

Si vogliono derivare dalla classe Persona le seguenti classi:

- Studente
- Lavoratore

La prima deve avere gli attributi aggiuntivi:

- Scuola
- Media voti

La seconda deve avere gli attributi aggiuntivi:

- Azienda
- Stipendio

Aggiungere tutti i metodi getter e setter relativi agli attributi aggiuntivi.

Inoltre effettuare l'overriding dei costruttori e del metodo str inserendo gli attributi aggiuntivi.

Provare le tre classi instanziando almeno un oggetto per classe e provando qualche metodo.

Quinto Esercizio

Creare una classe AritmeticaDue con attributi operando1 e operando2. Definire il costruttore utilizzando parametri con valori predefiniti e il metodo str.

Aggiungere due metodi uno che restituisca la differenza e l'altro il prodotto dei due operandi. Implementare un terzo metodo che permetta il confronto tra il risultato del prodotto di due oggetti AritmeticaDue (in sostanza indicare se il prodotto è maggiore di quello calcolato nell'oggetto AritmeticaDue passato come parametro).

Derivare dalla classe AritmeticaDue la classe AritmeticaTre aggiungendo l'attributo operando3. Ridefinire il costruttore, il metodo str e i tre metodi differenza, prodotto e confronto. Aggiungere un metodo per il calcolo della somma di tutti gli attributi.

Provare le classi e i metodi implementati.

Esercizi sul polimorfismo e sull'import

Sesto Esercizio: Polimorfismo con oggetti veicolo

Si definisca la seguente:

1. Classe base Veicolo con:

   - ➔ Costruttore che accetta marca e modello
   - ➔ Metodo accendi() che restituisce la stringa:"Il veicolo si avvia"
   - ➔ Metodo spegni() che restituisce una stringa:"Il veicolo si spegne"
   - ➔ Metodo __str__() per la rappresentazione testuale

Successivamente:

2. Classe Auto che estende Veicolo con:

   - ➔ Costruttore che accetta marca, modello e numero_porte
   - ➔ Override del metodo accendi() che restituisce: "L'auto si avvia con la chiave"
   - ➔ Override del metodo spegni() che restituisce: "L'auto si spegne girando la chiave"

3. Classe Moto che estende Veicolo con:

   - ➔ Costruttore che accetta marca, modello e cilindrata
   - ➔ Override del metodo accendi() che restituisce: "La moto si avvia premendo il pulsante"
   - ➔ Override del metodo spegni() che restituisce: "La moto si spegne con l'interruttore"

4. Classe Bicicletta che estende Veicolo con:

   - ➔ Costruttore che accetta marca, modello e tipo (es. "mountain bike", "city bike")
   - ➔ Override del metodo accendi() che restituisce: "La bicicletta è sempre pronta!"
   - ➔ Override del metodo spegni() che restituisce: "La bicicletta non si spegne mai"

5. Implementare la funzione gestisci_veicolo(veicolo) che:

   - ➔ Stampa le informazioni del veicolo
   - ➔ Chiama i metodi accendi() e spegni() sul veicolo ricevuto

6. Scrivere un programma principale che:

- ➔ Crea istanze di tutti i tipi di veicolo (parametri hard-coded)
- ➔ Le inserisce in una lista
- ➔ Itera sulla lista chiamando gestisci_veicolo() per ogni elemento

Esempio di output atteso:

```
Auto: Fiat Panda

L'auto si avvia con la chiave

L'auto si spegne girando la chiave

---

Moto: Honda CBR

La moto si avvia premendo il pulsante

La moto si spegne con l'interruttore

---

Bicicletta: Bianchi Mountain Bike

La bicicletta è sempre pronta!

La bicicletta non si spegne mai

---
```

Settimo Esercizio: Polimorfismo con operatori e collezioni eterogenee

Si vuole creare un programma che tratti i numeri complessi in forma algebrica e i vettori 2D. A tale scopo si definiscano le seguenti due classi:

1. Classe NumeroComplesso con:

- ➔ Costruttore che accetta parte_reale e parte_immaginaria (rispettivamente a e b)
- ➔ Metodo __str__() che restituisce la forma forma algebrica: "a + ib" (se b <0 allora “a - i|b|)
- ➔ Metodo __add__() per sommare due numeri complessi. Regola: (a+ib) + (c+id) = (a+c) + i(b+d)
- ➔ Metodo __mul__() per moltiplicare due numeri complessi. Regola: (a+ib)(c+id) = (ac - bd) + i(ad+bc)
- ➔ Metodo modulo() che calcola il modulo del numero complesso. Regola: |a+ib| = √(a²+b²)

2. Classe Vettore2D con:

- ➔ Costruttore che accetta x e y

- ➔ Metodo __str__() che restituisce la forma "(x, y)"
- ➔ Metodo __add__() per sommare due vettori. Regola: (a,b)+(c,d) = (a+c , b+d)
- ➔ Metodo __mul__() per moltiplicare un vettore per uno scalare (numero). Regola: (a,b)*n = (an , bn)
- ➔ Metodo modulo() che calcola la lunghezza (o modulo) del vettore. | (a,b) | = $\sqrt{(a^2+b^2)}$

3. Implementare la funzione stampa_operazioni(obj1, obj2) che:

- ➔ Stampa i due oggetti
- ➔ Stampa la loro somma
- ➔ Stampa il modulo di entrambi

4. Scrivere un programma principale che:

- ➔ Crea istanze di NumeroComplesso e Vettore2D
- ➔ Testa le operazioni su ciascun tipo
- ➔ Crea una lista mista contenente numeri complessi e vettori
- ➔ Itera sulla lista calcolando e stampando il modulo di ogni elemento

Esempio di output atteso:

```
Numero complesso: 3 + 4i, modulo: 5.0

Numero complesso: 1 - 2i, modulo: 2.236

Somma: 4 + 2i

---

Vettore: (5, 12), modulo: 13.0

Vettore: (3, 4), modulo: 5.0

Somma: (8, 16)

---

Collezione mista:

Elemento: 3 + 4i, modulo: 5.0

Elemento: (5, 12), modulo: 13.0

Elemento: 1 - 2i, modulo: 2.236

---
```

Ottavo Esercizio: Sistema modulare per gestione biblioteca [Facoltativo]

Si vuole progettare un programma per la gestione di una biblioteca utilizzando l'importazione di classi da file separati.

Dopo aver modellizzato le classi che intervengono nel programma, si preferisce organizzare il codice in tre file separati:

File 1: media.py

Definisce le seguenti classi:

Classe base Media con:

- ➔ Costruttore che accetta titolo, autore, anno
- ➔ Metodo get_info() che restituisce le informazioni base
- ➔ Metodo __str__() per la rappresentazione

Classe Libro che eredita da Media con:

- ➔ Costruttore che accetta anche numero_pagine
- ➔ Override di get_info() che include il numero di pagine
- ➔ Metodo tempo_lettura() che calcola il tempo di lettura (1 pagina = 2 minuti)

Classe DVD che eredita da Media con:

- ➔ Costruttore che accetta anche durata_minuti
- ➔ Override di get_info() che include la durata
- ➔ Metodo tempo_visione() che restituisce la durata in formato "ore:minuti"

File 2: biblioteca.py

Definisce la classe:

Classe Biblioteca con:

- ➔ Costruttore che inizializza una lista vuota di media
- ➔ Metodo aggiungi_media(media) per aggiungere un elemento
- ➔ Metodo rimuovi_media(titolo) per rimuovere per titolo
- ➔ Metodo cerca_per_autore(autore) che restituisce una lista
- ➔ Metodo stampa_catalogo() che stampa tutti i media
- ➔ Metodo get_statistiche() che restituisce il numero di libri e DVD

- ➔ File 3: main.py

Programma principale che:

- ➔ Importa le classi necessarie dai moduli media e biblioteca

- ➔ Crea una biblioteca
- ➔ Aggiunge diversi libri e DVD
- ➔ Testa tutti i metodi implementati
- ➔ Stampa il catalogo completo e le statistiche

Struttura cartelle richiesta:

```
progetto/
|-- main.py
|-- media.py
|-- biblioteca.py
```

N.B. Non è necessario effettuare delle procedure di input da tastiera, le classi devono essere istanziate in modalità hard-coded.

Esempio di output atteso:

```
=== CATALOGO BIBLIOTECA ===
Libro: "1984" di George Orwell (1949) - 328 pagine
Tempo di lettura stimato: 656 minuti (10 ore e 56 minuti)
---
DVD: "Inception" di Christopher Nolan (2010) - 148 minuti
Tempo di visione: 2 ore e 28 minuti
---
Libro: "Il Nome della Rosa" di Umberto Eco (1980) - 512 pagine
Tempo di lettura stimato: 1024 minuti (17 ore e 4 minuti)

=== STATISTICHE ===
Libri: 2
DVD: 1
Totale media: 3

=== RICERCA PER AUTORE "Orwell" ===
```

```
Libro: "1984" di George Orwell (1949) - 328 pagine
```